

Exercício Prático: Gerenciamento e Ordenação de uma Fila Circular de Processos do sistema Operacional (2.0 Pontos)

Regras de aceitação da entrega: Implementação deve ser em Java, os programas devem estar devidamente compilados e executando sem erros, será levado em consideração os melhores códigos ou seja, com a lógica correta.

Contexto: Imagine que você está desenvolvendo um sistema operacional simplificado onde os processos são executados em um ciclo contínuo. Para gerenciar esses processos, você utilizará uma **Lista Encadeada Circular**, onde o último processo na fila aponta de volta para o primeiro, criando um "círculo". Além disso, em certos momentos, você precisará ordenar esses processos com base em sua prioridade usando o algoritmo **Merge Sort**, que é uma escolha eficiente para listas encadeadas e intrinsecamente recursivo.

Objetivos de Aprendizagem:

- Reforçar o entendimento das **listas encadeadas circulares** e suas operações.
- Praticar a **implementação de operações** em listas encadeadas circulares (inserção, remoção, travessia).
- Desenvolver a capacidade de adaptar e implementar **algoritmos de ordenação (Merge Sort)** para listas encadeadas.
- Aplicar o conceito de **recursividade** na resolução de problemas (especialmente no Merge Sort).
-

Descrição do Problema:

Você deve criar uma classe **Processo** (ou similar) e, em seguida, uma classe **ListaCircularDeProcessos** que implemente as funcionalidades abaixo.

Parte 1: Implementação da Lista Encadeada Circular de Processos

1. Estrutura do Nó (Processo):

- Crie uma classe **NoProcesso** que represente um nó na lista. Cada nó deve conter:
 - **id** (inteiro): Identificador único do processo.
 - **prioridade** (inteiro): Um valor numérico que indica a prioridade do processo (ex: 1 = alta, 5 = baixa).
 - **descricao** (String): Uma descrição do processo.
 - **proximo** (referência a **NoProcesso**): Ponteiro para o próximo nó na lista.

2. Classe `ListaCircularDeProcessos`:

- Crie uma classe `ListaCircularDeProcessos` que gerencie a lista circular. Ela deve ter um ponteiro `head` (cabeça) que aponta para o primeiro nó da lista.
- Implemente os seguintes métodos:
 - `inserirProcesso(id, prioridade, descricao)`: Adiciona um novo processo à lista. Pode ser no final da lista, mantendo a circularidade. Lembre-se que o último nó aponta de volta para o primeiro.
 - `removerProcesso(id)`: Remove um processo específico da lista com base em seu `id`.
 - `percorrerEImprimir()`: Percorre a lista e imprime o `id` e a `prioridade e descricao` de cada processo. Lembre-se que, em uma lista circular, a condição de parada do loop de travessia deve ser quando o nó atual for novamente o nó inicial (`head`), em vez de verificar `NULL`.

Parte 2: Ordenação da Lista de Processos usando Merge Sort Recursivo

1. Preparação para Ordenação:

- Para simplificar a aplicação do Merge Sort (que é geralmente mais intuitivo em listas não circulares), você pode optar por "quebrar" temporariamente a circularidade da lista antes de ordenar e restabelecê-la depois, ou adaptar o algoritmo para gerenciar o ponteiro `head` de forma que a lista se comporte como uma lista simplesmente encadeada durante a ordenação.

2. Implementação do Merge Sort:

- Adicione um método `ordenarPorPrioridade()` à sua classe `ListaCircularDeProcessos`. Este método deve utilizar o algoritmo **Merge Sort** para ordenar os processos com base em sua `prioridade` (prioridade menor significa mais urgente, ou seja, ordem crescente).
- O Merge Sort é um algoritmo de "divisão e conquista". Ele requer as seguintes sub-rotinas (que provavelmente serão métodos auxiliares e recursivos):
 - `dividirLista(head)`: Divide a lista encadeada (ou uma sub-lista) em duas metades. Use a técnica dos "ponteiros rápido e lento" (`fast and slow pointer`) para encontrar o ponto médio de forma eficiente.
 - `merge(lista1, lista2)`: Mescla duas sub-listas ordenadas em uma única lista ordenada, ajustando apenas os ponteiros `proximo` dos nós.

- `mergeSort(head)`: Esta é a função recursiva principal. Ela deve:
 - Ter um **caso base**: Se a lista for vazia ou tiver apenas um elemento, ela já está ordenada.
 - Ter um **passo recursivo**: Dividir a lista em duas metades, chamar `mergeSort` recursivamente em cada metade e, em seguida, mesclar as duas metades ordenadas.

Exemplo de Execução (Teste):

1. Crie uma instância de `ListaCircularDeProcessos`.
2. Insira alguns processos com diferentes `ids`, `prioridades` e `descricao` (ex: (1, 3, "executar calculadora"), (2, 1, "executar navegador Chrome"), (3, 5, "executar programa em java"), (4, 2, "executar o Word")).
3. Imprima a lista antes da ordenação para ver a ordem inicial.
4. Chame o método `ordenarPorPrioridade()`.
5. Imprima a lista após a ordenação para verificar se os processos estão ordenados corretamente por prioridade (ex: (2, 1), (4, 2), (1, 3), (3, 5), com as suas descrições).
6. Remova um processo e imprima novamente.